

● SESIÓN 1 :Spring Boot REST – Fundamentos (sin base de datos)

Proyecto: **Books API (in-memory)**

🎯 OBJETIVOS DE LA CLASE

Fundamentos REST

Entiende **qué es una API REST**

HTTP y CRUD

Conoce los **métodos HTTP y CRUD**

Spring Boot

Sabe crear un proyecto Spring Boot

Endpoints

Implementa **endpoints reales**

Parámetros

Maneja **Path Variables y Query Params**

Swagger

Usa **Swagger** para probar la API

JSON ↔ Java

Comprende el flujo **JSON ↔ Java (Jackson)**

● BLOQUE 0 – ¿Qué vamos a crear?

Mensaje literal para clase

"Vamos a crear una **API REST de libros**. No hay interfaz gráfica. Solo URLs, JSON y Spring Boot."

Funcionalidad final

- Obtener todos los libros
- Obtener un libro concreto
- Filtrar libros
- Crear, actualizar y borrar libros

📄 ⚠️ **Aclaración importante:**

"Los datos estarán **en memoria**, no en base de datos. Eso lo veremos más adelante."

BLOQUE 1 – REST + CRUD + HTTP

Concepto clave

CRUD = operaciones básicas sobre recursos

Operación	HTTP	Qué hace
Create	POST	Crear recurso
Read	GET	Leer recurso
Update	PUT	Actualizar
Delete	DELETE	Eliminar

Frase clave:

"REST no es Spring. Spring solo lo implementa."

BLOQUE 2 – Crear el proyecto (Spring Initializr)

Configuración

- Project: **Maven**
- Language: **Java**
- Java: **17-21-23-25**
- Packaging: **Jar**
- Dependencies:
 - Spring Web
 - Springdoc OpenAPI UI (Swagger)

Explicación breve:

- Spring Web → REST
- Swagger → probar la API sin Postman



BLOQUE 3 – Primer endpoint GET (Hello World)

 controller/BookController.java

```
package com.example.books.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class BookController {

    @GetMapping("/api/hello")
    public String hello() {
        return "Hello Spring Boot!";
    }
}
```

Qué estamos viendo:

@RestController

@GetMapping

Qué es un endpoint

URL → método Java

Probar

 <http://localhost:8080/api/hello>

BLOQUE 4 – JSON ↔ POJO (Jackson)

Concepto clave

"Spring convierte automáticamente JSON ↔ objetos Java."

Eso se llama:



Data Binding



Serialization / Deserialization



Jackson (por debajo)

BLOQUE 5 – Crear el modelo Book

 model/Book.java

```
package com.example.books.model;

public class Book {
    private String title;
    private String author;
    private String category;

    public Book(String title, String author, String category) {
        this.title = title;
        this.author = author;
        this.category = category;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getCategory() {
        return category;
    }
}
```

 **Mensaje clave:**

"Este objeto **no es REST**, es Java puro."

BLOQUE 6 – REST Controller real (/api/books)

 BookController.java

```
package com.example.books.controller;

import com.example.books.model.Book;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final List<Book> books = List.of(
        new Book("Clean Code", "Robert C. Martin", "programming"),
        new Book("Effective Java", "Joshua Bloch", "programming"),
        new Book("Sapiens", "Yuval Noah Harari", "history")
    );

    @GetMapping
    public List<Book> getBooks() {
        return books;
    }
}
```

BLOQUE 7 – Path Variables

Problema

"Quiero **un solo libro**, no todos."

Endpoint

```
@GetMapping("/{index}")
public Book getBookByIndex(@PathVariable int index) {
    return books.get(index);
}
```

Probar

- /api/books/0
- /api/books/1

Aclaración:

"Usamos índice SOLO para aprender Path Variables."

BLOQUE 8 – Query Params

Problema

"Quiero filtrar libros por categoría."

Probar

- /api/books?category=programming
- /api/books?category=history

Mensaje clave:

"Path → identificar
Query → filtrar"

Endpoint

```
@GetMapping
public List<Book> getBooks(
    @RequestParam(required = false)
    String category) {

    if (category == null) {
        return books;
    }

    return books.stream()
        .filter(book ->
            book.getCategory()
                .equalsIgnoreCase(category))
        .toList();
}
```



BLOQUE 9 – POST / PUT / DELETE



POST – Crear libro

```
@PostMapping
public void createBook(
    @RequestBody Book newBook) {
    books.add(newBook);
}
```



PUT – Actualizar

```
@PutMapping("/{index}")
public void updateBook(
    @PathVariable int index,
    @RequestBody Book book) {
    books.set(index, book);
}
```



DELETE – Eliminar

```
@DeleteMapping("/{index}")
public void deleteBook(
    @PathVariable int index) {
    books.remove(index);
}
```



Mensaje clave:

"HTTP define la intención, no el método Java."

BLOQUE 10 – Swagger

Dependencia (Maven)

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>
```

Acceso

👉 <http://localhost:8080/swagger-ui/index.html>

Configuración

```
springdoc.swagger-ui.path=/docs
```

Acceso

👉 <http://localhost:8080/docs>

Qué vemos

- Endpoints
- Métodos HTTP
- Request body
- Probar sin Postman

💡 Frase clave:

"Swagger es tu mejor amigo para desarrollar APIs."



BLOQUE 11 – Cierre de la sesión

Lo que debes llevarte claro



Qué es REST



Cómo Spring crea APIs



Qué es un endpoint



Qué hace Jackson



Diferencia Path vs Query



CRUD completo



Swagger como herramienta